# Git_Challenge-1

1. **Resolve Merge Conflicts**
    - Create a merge conflict intentionally (two users editing the same line).
    - Resolve the conflict and push the changes.

Step 1: Create a merge conflict

# Suppose you have two branches: **master** and **release**

Both branches modify the same line in **sample** file

**master** changes it to “This is a git Task”

**release** changes it to “This is c git challenge”

# Now merge **release** into **master**

**: git checkout master**

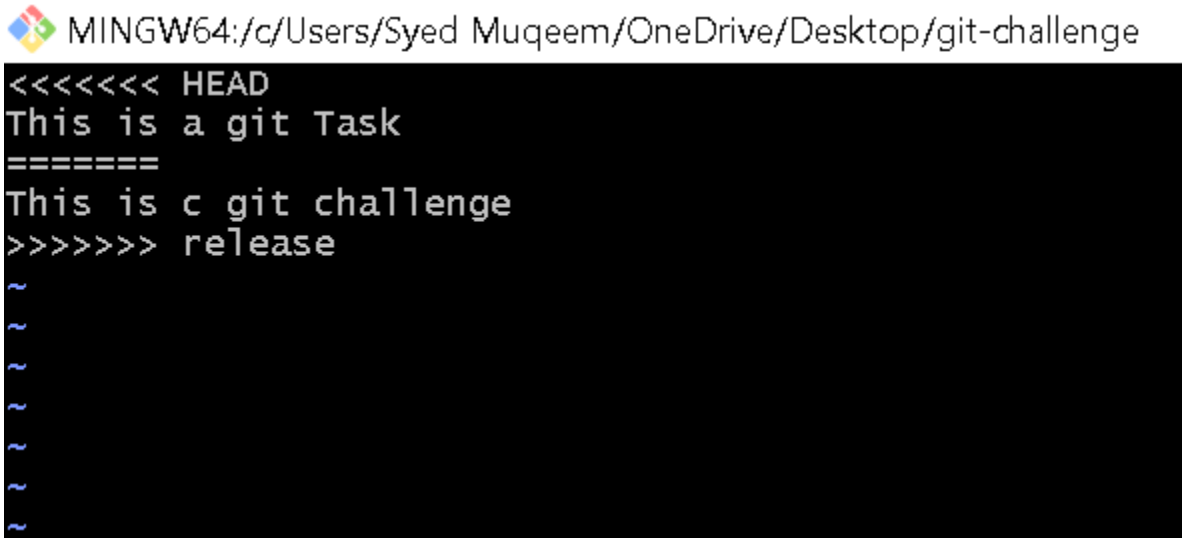
**git merge release**

**Step 2: Check the conflict**

**: git status**

# Git shows which files have conflicts

**#** Open the conflicted file. You may see



```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ cat sample
This is a git Task

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git commit -m "added text to sample file in master branch"
[master b978d31] added text to sample file in master branch
 1 file changed, 1 insertion(+)

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git merge release
Auto-merging sample
CONFLICT (content): Merge conflict in sample
Automatic merge failed; fix conflicts and then commit the result.

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|MERGING)
$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   sample

no changes added to commit (use "git add" and/or "git commit -a")

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|MERGING)
$ vi sample

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|MERGING)
$
```

#Step 3: Resolve the conflict

# Decide which content you want to keep, or combine both versions.

**Ex: This is a git Task/challenge**

# Save the vi editor file

Step 4: Stage the resolved file

: git add .

: git commit -m "modified sample conflict file in master"

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|MERGING)
$ vi sample

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|MERGING)
$ git status
On branch master
All conflicts fixed but you are still merging.
  (use "git commit" to conclude merge)

Changes to be committed:
        modified:   sample

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   sample


Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|MERGING)
$ git add .

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|MERGING)
$ git commit -m "modified sample file for git conflict in master"
[master 6c253b9] modified sample file for git conflict in master

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git merge release
Already up to date.

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ cat sample

This is a git Task/challenge
```

2. **Recover Deleted Branch**
    - Delete a local branch and then recover it using the reflog.

# Before running the delete command, you must switch out of the branch you want to delete.

# **Safe deletion (Recommended):** This will only delete the branch if you have already merged your changes into another branch.

git checkout **main**

git branch -d **release**

MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$  git branch -d release
Deleted branch release (was b67f585).

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git branch
* master


Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$
```

**Step 1: Check recent Git history**

: git reflog

# Find the commit where your deleted branch was pointing

# Copy the commit ID of the deleted branch's latest commit

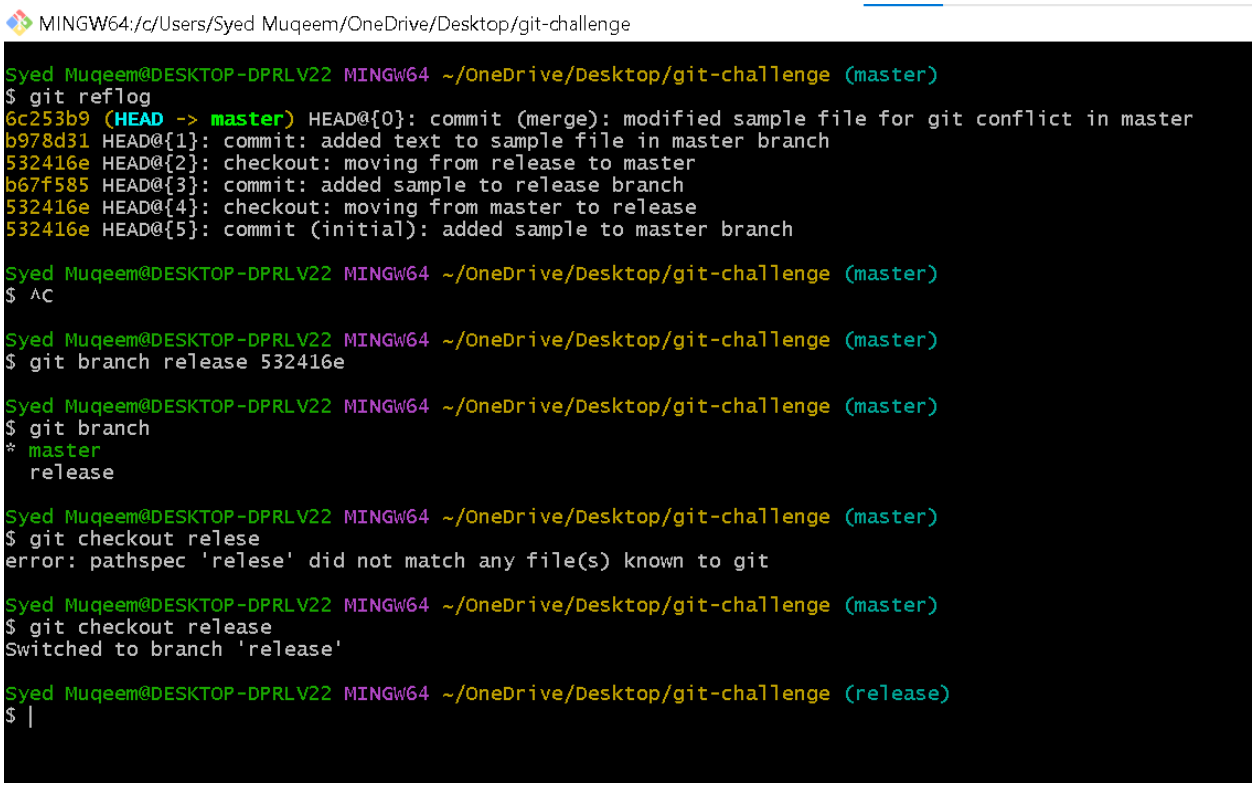
Step 2: Recreate the deleted branch

: git branch release **532416e**

Step 3: Switch to the recovered branch

**: git branch**

: git checkout **release**




3. **Undo Wrong Push**
    - Push a wrong commit to GitHub, then undo it without losing history.

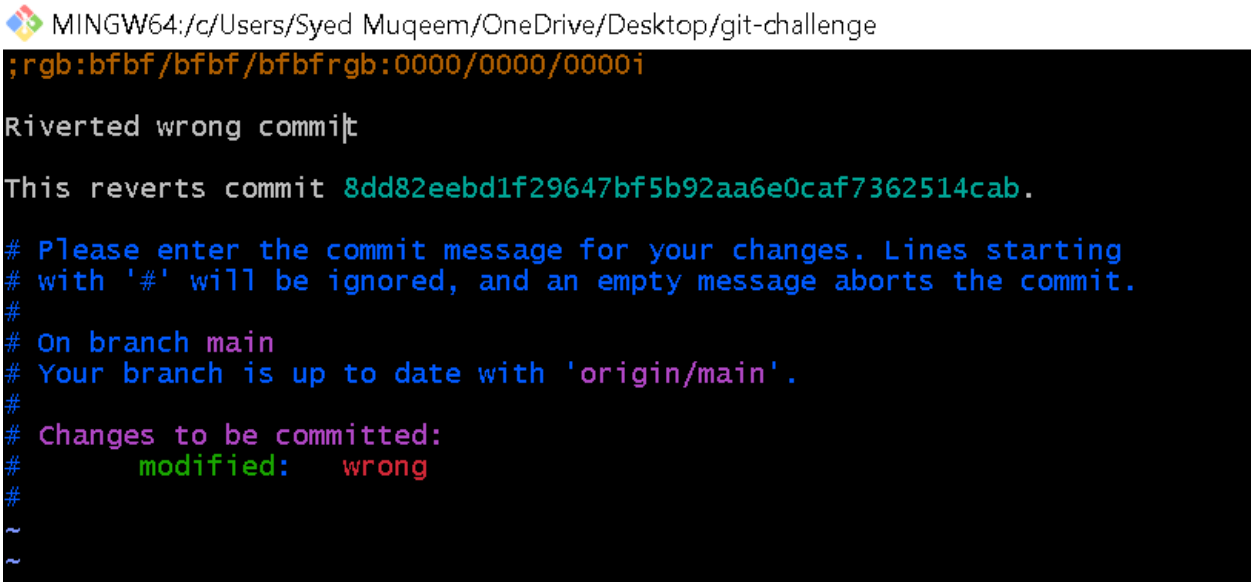
```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ touch wrong

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        wrong

nothing added to commit but untracked files present (use "git add" to track)

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git add .

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git commit -m "added file wrong into master"
[master 4b3dd3e] added file wrong into master
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 wrong
```




4. **Amend a Commit**
    - Make a commit, then add a missing file to it using `git commit --amend`.

MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge

```
Revert completed for the file wrong commit

This reverts commit 8dd82eebd1f29647bf5b92aa6e0caf7362514cab.

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Mon Oct 26 15:24:27 2026 +0530
#
# On branch main
# Your branch is ahead of 'origin/main' by 1 commit.
#   (use "git push" to publish your local commits)
#
# Changes to be committed:
#       modified:   wrong
#
~
~
~
~
~
```

MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git commit --amend
[main 33a122c] Revert completed for the file wrong commit
 Date: Mon Oct 26 15:24:27 2026 +0530
 1 file changed, 1 deletion(-)

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git log --oneline
33a122c (HEAD -> main) Revert completed for the file wrong commit
c301962 (origin/main, origin/HEAD) ;rgb:bfbf/bfbf/bfbfrgb:0000/0000/0000
8dd82ee added text to file wrong in main branch
ce9c255 Add welcome message to Hello file
4b3dd3e added file wrong into master
6c253b9 modified sample file for git conflict in master
b978d31 added text to sample file in master branch
b67f585 added sample to release branch
532416e (release) added sample to master branch

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$
```

5. **Cherry-pick a Commit**
    - Take a specific commit from one branch and apply it to another branch.

I have two branches:

- main— contains the commit you need.
- `release`— where you want to apply that commit.

Step 1: Find the commit ID

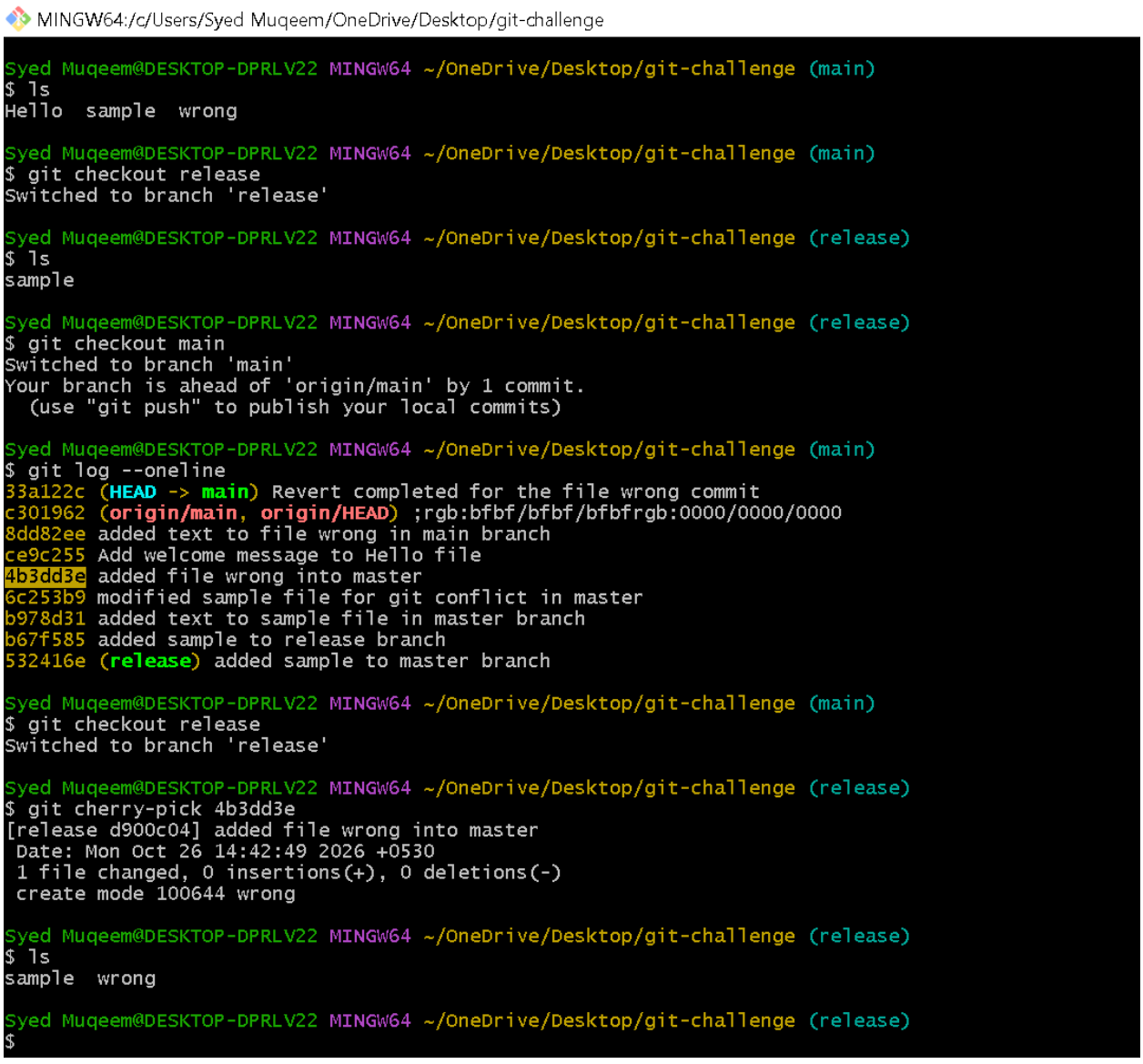
git checkout Branch-git

git log --oneline

# Copy the commit ID you want,

Step 2: Switch to the target branch and apply

**git cherry-pick 4b3dd3e**




6. **Interactive Rebase**
    - Reorder and squash multiple commits into a single clean commit.

```
MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge
pick ce9c255 # Add welcome message to Hello file
squash 33a122c # Revert completed for the file wrong commit

# Rebase 8dd82ee..33a122c onto 8dd82ee (2 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                    commit's log message, unless -C is used, in which case
#                    keep only this commit's message; -c is same as -C but
#                    opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
#         create a merge commit using the original merge commit's
#         message (or the oneline, if no original merge commit was
#         specified); use -c <commit> to reword the commit message
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
#                       to this position in the new commits. The <ref> is
#                       updated at the end of the rebase
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
~
~
```

```
MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git rebase -i HEAD~2
[detached HEAD 9fc4223] ;rgb:bfbf/bfbf/bfbfrgb:0000/0000/0000
 Date: Mon Oct 26 14:49:57 2026 +0530
 2 files changed, 1 insertion(+), 1 deletion(-)
 create mode 100644 Hello
Successfully rebased and updated refs/heads/main.

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git log
commit 9fc4223153a2d6c7b6666b9201adcf2ff0ff9c42 (HEAD -> main)
Author: muqeem9256-bot <muqeem9256@gmail.com>
Date:   Mon Oct 26 14:49:57 2026 +0530

    ;rgb:bfbf/bfbf/bfbfrgb:0000/0000/0000

    Add welcome message to Hello file

    Revert completed for the file wrong commit

    This reverts commit 8dd82eebd1f29647bf5b92aa6e0caf7362514cab.
```

7. **Tagging & Release**
    - Create a version tag (v1.0), push it to GitHub, then delete and restore it.

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git tag -a v1.0 -m "Version 1.0 release"

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git push origin v1.0
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 559 bytes | 279.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:muqeem9256-bot/Challenge.git
 * [new tag]         v1.0 -> v1.0

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git tag
v1.0
```

Step 3: Delete the tag locally

git tag -d v1.0

# This deletes the tag from your local computer only. It does not delete the GitHub tag.

Step 4: Delete the tag from GitHub

git push origin --delete v1.0

MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git tag -d v1.0
Deleted tag 'v1.0' (was 9348cc2)

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git tag

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git push origin --delete v1.0
To github.com:muqeem9256-bot/Challenge.git
 - [deleted]         v1.0
```

Step 5: Restore the tag

# Create the tag again on the same commit you originally tagged.

If you saved the original commit ID, use it.

**git tag -a v1.0 9fc42 -m "Version 1.0 release"**

**git push origin v1.0**

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git tag -a v1.0 9fc42 -m "Version 1.0 release"

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git push origin v1.0
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (5/5), 560 bytes | 280.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:muqeem9256-bot/Challenge.git
 * [new tag]         v1.0 -> v1.0

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (main)
$ git tag
v1.0
```

8. **Clone with Sparse Checkout**
    - Clone only a subdirectory of a repo using sparse checkout.

MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge
$ git init
Initialized empty Git repository in C:/Users/Syed Muqeem/OneDrive/Desktop/git-challenge/.git/

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ ls
sample  wrong

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git remote add -f origin https://github.com/imrankhanmohammad257/devops-cloud-docs.git
Updating origin
remote: Enumerating objects: 148, done.
remote: Total 148 (delta 0), reused 0 (delta 0), pack-reused 148 (from 1)
Receiving objects: 100% (148/148), 105.77 MiB | 2.97 MiB/s, done.
Resolving deltas: 100% (27/27), done.
From https://github.com/imrankhanmohammad257/devops-cloud-docs
 * [new branch]      main       -> origin/main

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ ls
sample  wrong

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master)
$ git config core.sparseCheckout true

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|SPARSE)
$ git sparse-checkout
error: need a subcommand
usage: git sparse-checkout (init | list | set | add | reapply | disable | check-rules | clean) [<options>]


Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (master|SPARSE)
$ git sparse-checkout set liunx
```




9. **Reset vs Revert Challenge**
    - Demonstrate the difference between `git reset --hard` and `git revert` in a repo.

# In Git, both `git reset --hard` and `git revert` are used to undo changes, but the main difference is how they handle commit history.

### 1. git-- hard reset

Moves your current branch backward to a specified commit and discards changes from your working directory and staging area.

Ex: git reset --hard HEAD~1

# This moves the current branch back by one commit.

Before: A → B → C (HEAD)

After: A → B (HEAD)

#Commit `C` is removed from the current branch's history.

#Changes introduced by `C` are discarded from your working directory.

#You may be able to recover the commit using `git reflog`

### 2. git revert

Creates a new commit that reverses the changes introduced by an earlier commit.

Ex: git revert HEAD

Before: A → B → C (HEAD)

After: A → B → C → D (HEAD)

Here, `D` is the new revert commit that undoes the changes from `C`.

# The original commit `C` remains in history.

# A new commit reverses its changes.

# It is generally safer for commits already pushed to GitHub.

Main differences:

| **Feature** | **git reset --hard** | **git revert** |
|---|---|---|
| Creates a new commit | No | Yes |
| Keeps original commit in current branch history | No | Yes |
| Discards uncommitted changes | Yes | Usually no, but conflicts can occur |
| Rewrites branch history | Yes | No |
| Best for pushed commits | Usually avoid | Recommended |
| Best for local mistakes | Often useful | Also useful |

10. **Detached HEAD Challenge**
    - Checkout a specific commit (detached HEAD state) and create a new branch from it.

Step 1: check the Commits

git log --oneline

Step 2: checkout the Specific commit

git checkout **915522c**

**# Now you are on Detached HEAD state, means not in any branch**

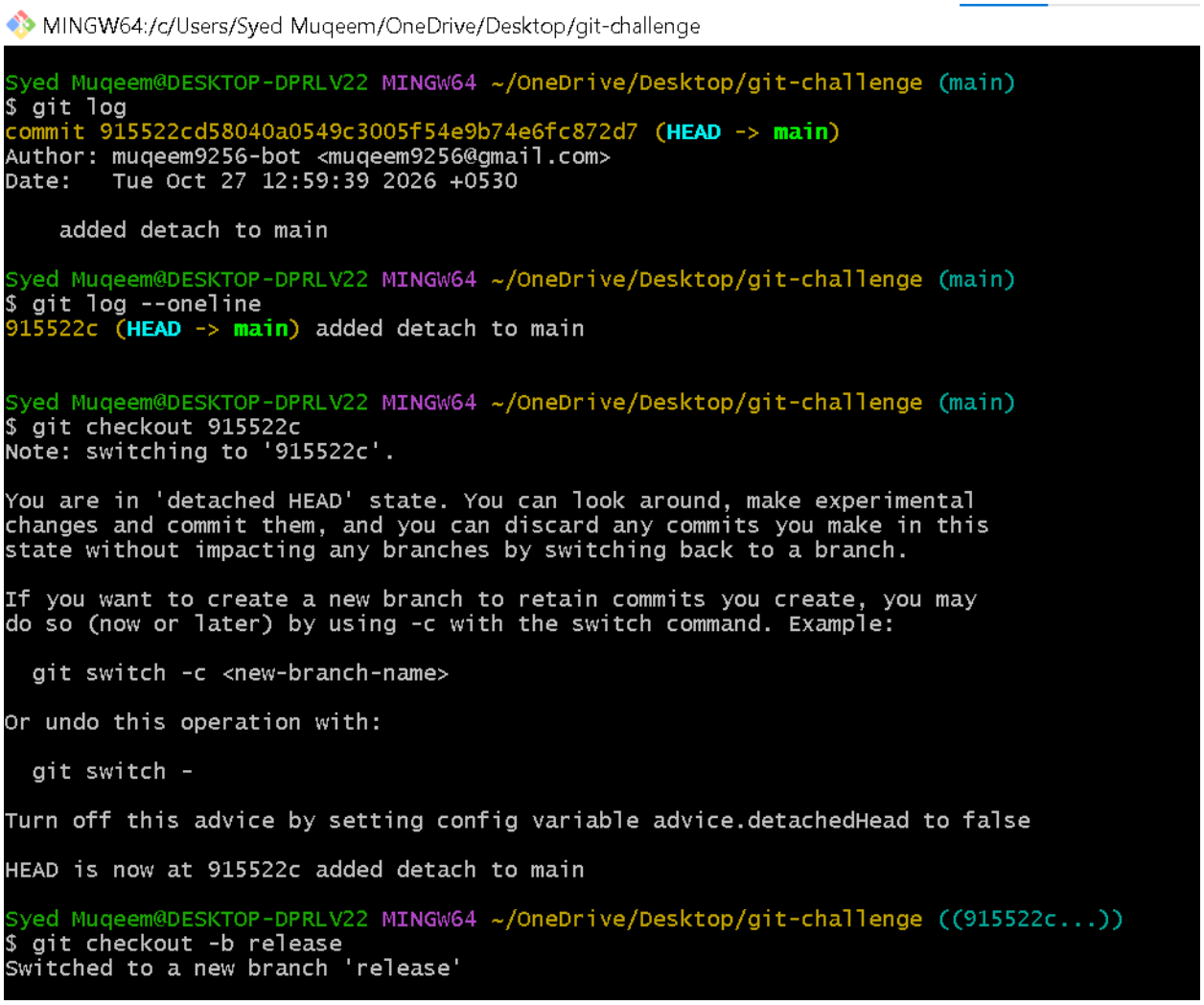
Step 3: From the that commit create a new branch.

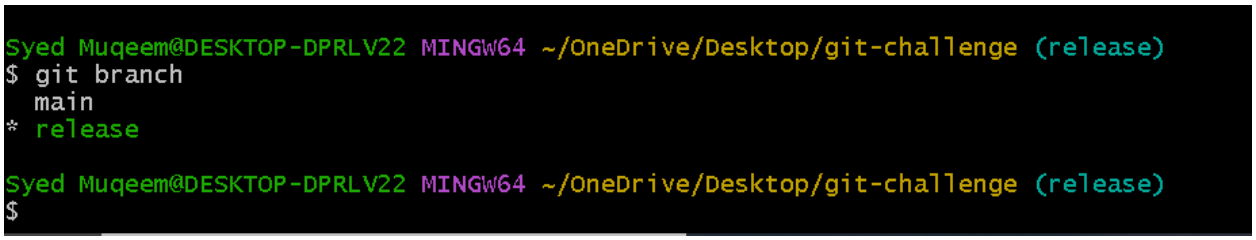
git checkout -b **release**

**Step 4: Verify**

**Git branch**

**Git status**




11. **Git Hooks Challenge**
    - Configure a `pre-commit` hook to reject commits without a message format (e.g., must start with `JIRA-XXX`).

Step 1: Go to the project directory

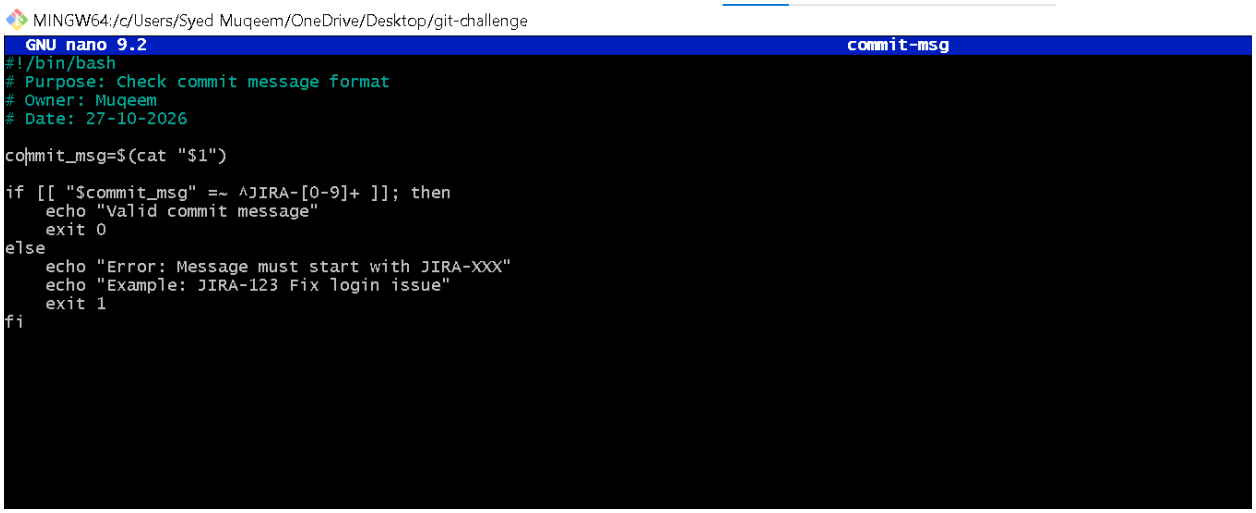
cd ~/OneDrive/Desktop/git-challenge

Step 2: Open the Git hooks directory

cd .git/hooks

Step 3: Create the `commit-msg` fil

nano commit-msg




Step 4: Give execution permission

chmod 755 commit-msg

Step 5: Return to the project directory

cd ../..

Step 6: Create a test file and stage it

echo "Testing Git hook" >> test.txt

git add test.txt

Step 7: Test a valid commit message

git commit -m "JIRA-123 Fix login issue"

# Result: The commit was accepted because the message followed the required format.

Step 8: Create another change

echo "Second test" >> test.txt

git add test.txt

Step 9: Test an invalid commit message

git commit -m "Fixed login issue"

# Result: The commit was rejected, and Git displayed

# Error: Message must start with JIRA-XXX

Example: JIRA-123 Fix login issue

MINGW64:/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ cd .git/hooks

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge/.git/hooks (GIT_DIR!)
$ nano commit-msg

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge/.git/hooks (GIT_DIR!)
$ chmod 755 commit-msg

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge/.git/hooks (GIT_DIR!)
$ git commit -m "JIRA-123 Fix login issue"
fatal: this operation must be run in a work tree
```

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge/.git/hooks (GIT_DIR!)
$ cd ../..

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ ls
detach  sample  wrong

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ pwd
git status
/c/Users/Syed Muqeem/OneDrive/Desktop/git-challenge
On branch release
nothing to commit, working tree clean

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ git commit -m "JIRA-123 Fix login issue"
On branch release
nothing to commit, working tree clean

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ echo "Testing Git hook" >> test.txt
git add test.txt
warning: in the working copy of 'test.txt', LF will be replaced by CRLF the next time Git touches it

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ git commit -m "JIRA-123 Fix login issue"
Valid commit message

[release 872c2df] JIRA-123 Fix login issue
 1 file changed, 1 insertion(+)
 create mode 100644 test.txt

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ echo "Second test" >> test.txt
git add test.txt
warning: in the working copy of 'test.txt', LF will be replaced by CRLF the next time Git touches it

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (release)
$ git commit -m "Fixed login issue"
Error: Message must start with JIRA-XXX
Example: JIRA-123 Fix login issue
```

12. **Squash Merge vs Rebase Merge**
    - Show the difference between squash merge and rebase merge with evidence.

# Git: Squash Merge vs Rebase Merge

Both methods help integrate changes from one branch into another, but they handle commit history differently.

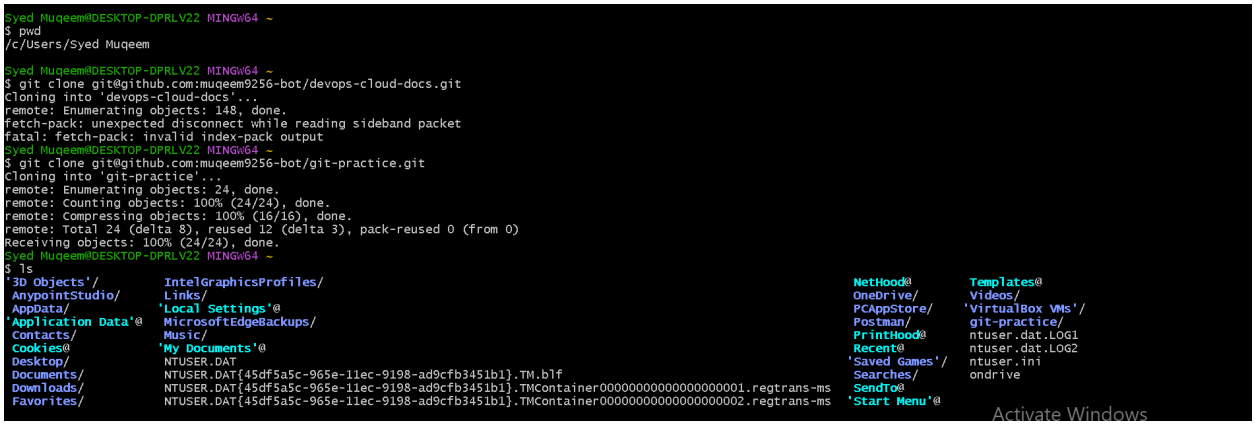
1.Squash merge: Combines multiple commits into one new commit on the target branch.

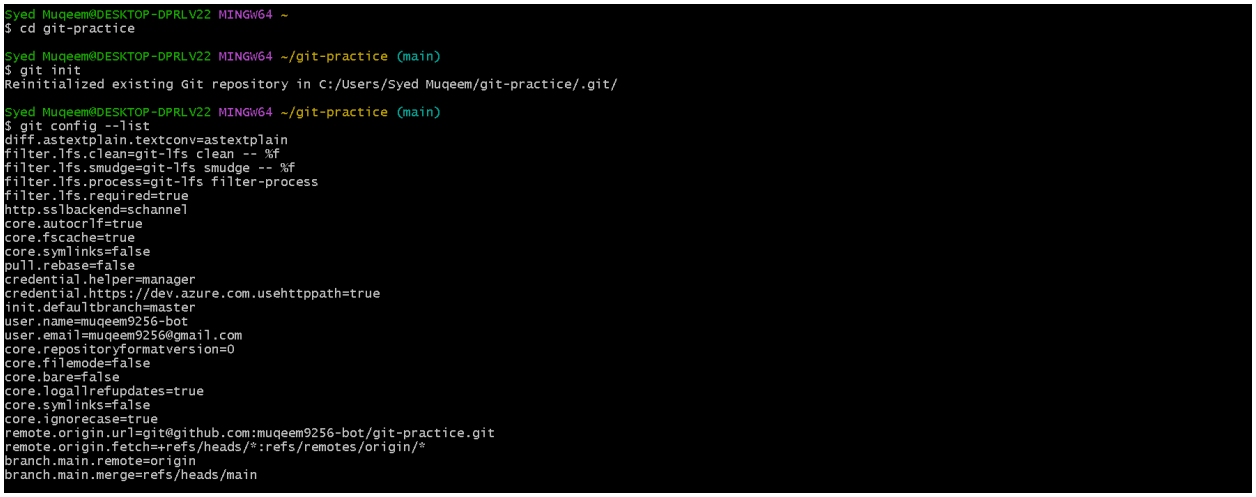
2.Rebase merge: Replays the individual commits on top of the target branch, creating a linear history without a merge commit.

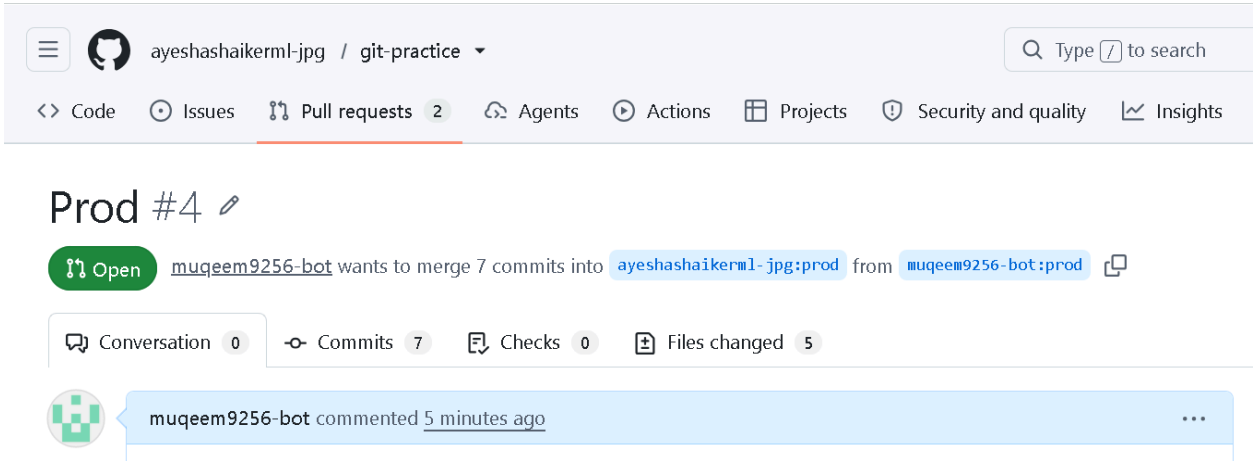
# Difference:

| Feature | Squash merge | Rebase merge |
|---|---|---|
| Number of commits on target branch | One combined commit | Individual commits preserved |
| Commit history | Short and clean | Linear and detailed |
| Original commit messages | Combined into one message | Preserved individually |
| Creates a merge commit | No | No |
| Changes commit IDs | Creates a new squash commit | Replayed commits get new IDs |

13. **Fork & Pull Request Workflow**
    - Fork a repo, make a change, and submit a pull request to the original repo.



```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (main)
$ git branch
* main

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (main)
$ git checkout -b prod
Switched to a new branch 'prod'

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (prod)
$ touch cat

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (prod)
$ git add .

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (prod)
$ git commit -m "added cat"
[prod 544eeb6] added cat
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 cat

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (prod)
$ git remote -v
origin  git@github.com:muqeem9256-bot/git-practice.git (fetch)
origin  git@github.com:muqeem9256-bot/git-practice.git (push)

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (prod)
$ git push -u origin prod
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 225 bytes | 225.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'prod' on GitHub by visiting:
remote:      https://github.com/muqeem9256-bot/git-practice/pull/new/prod
remote:
        To github.com:muqeem9256-bot/git-practice.git
 * [new branch]      prod -> prod
branch 'prod' set up to track 'origin/prod'.

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/git-practice (prod)
$
```

14. **Recover Lost Commit**
    - Commit something, reset hard, and then recover it using `git reflog`.

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ touch file

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git add .
warning: in the working copy of 'commit-msg', LF will be replaced by CRLF the next time Git touches
it

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git commit -m "added"
Error: Message must start with JIRA-XXX
Example: JIRA-123 Fix login issue

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git push -u origin prod
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (7/7), 535 bytes | 133.00 KiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
remote:
remote: Create a pull request for 'prod' on GitHub by visiting:
remote:      https://github.com/muqeem9256-bot/Challenge/pull/new/prod
remote:
To github.com:muqeem9256-bot/Challenge.git
 * [new branch]      prod -> prod
branch 'prod' set up to track 'origin/prod'.

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git log
commit 872c2df66f0bc94209b121c13a1b5a6a8927d2fe (HEAD -> prod, origin/prod, release)
Author: muqeem9256-bot <muqeem9256@gmail.com>
Date:   Tue Oct 27 13:21:42 2026 +0530

    JIRA-123 Fix login issue

commit 915522cd58040a0549c3005f54e9b74e6fc872d7 (main)
Author: muqeem9256-bot <muqeem9256@gmail.com>
Date:   Tue Oct 27 12:59:39 2026 +0530

    added detach to main
```

```
Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git reset --hard Head~1
HEAD is now at 915522c added detach to main

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git reflog
915522c (HEAD -> prod, main) HEAD@{0}: reset: moving to Head~1
872c2df (origin/prod, release) HEAD@{1}: checkout: moving from release to prod
872c2df (origin/prod, release) HEAD@{2}: commit: JIRA-123 Fix login issue
915522c (HEAD -> prod, main) HEAD@{3}: checkout: moving from 915522cd58040a0549c3005f54e9b74e6fc872d7 to release
915522c (HEAD -> prod, main) HEAD@{4}: checkout: moving from main to 915522c
915522c (HEAD -> prod, main) HEAD@{5}: commit (initial): added detach to main

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git reset --hard 915522c
HEAD is now at 915522c added detach to main

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$ git log
commit 915522cd58040a0549c3005f54e9b74e6fc872d7 (HEAD -> prod, main)
Author: muqeem9256-bot <muqeem9256@gmail.com>
Date:   Tue Oct 27 12:59:39 2026 +0530

    added detach to main

Syed Muqeem@DESKTOP-DPRLV22 MINGW64 ~/OneDrive/Desktop/git-challenge (prod)
$
```